ASSIGNMENT 9.5

2303A51930

BATCH-30

Problem 1: String Utilities Function
Consider the following Python function:
def reverse_string(text):
return text[::-1]
Task:
1. Write documentation in:
o (a) Docstring
o (b) Inline comments
o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string
Library

CODE:

```
# Write documentation in:
#  Docstring
#  Inline comments
#  Google-style documentation
def reverse_string(text):
    """
    Reverses the given string.

    Args:
        text (str): The string to be reversed.

    Returns:
        str: The reversed string.
    """
    # Use slicing to reverse the string
    return text[::-1]
#COMPARE THE THREE DOCUMENTATION METHODS:
# 1. Docstring: This method provides a clear and concise description of the function, its parameters, and its return value. It is typically used for documenting functions, classes, and modules in Python.
# 2. Inline comments: These are comments placed within the code to explain specific lines or sections of code. They are useful for providing additional context or clarifying complex logic, but they can make the code harder to read if overused.
# 3. Google-style documentation: This method follows a specific format for documenting functions, classes, and modules. It includes sections for Args, Returns, Raises, and Examples. It is widely used in the Python community and provides a standardized way of documenting code.
# In summary, docstrings are best for providing an overall description of a function or class, inline comments are useful for explaining specific lines of code, and Google-style documentation provides a structured format for documenting code in a consistent way.
```

RECOMMEND SUITABLE DOCUMENTATION METHOD:
For the given function, the most suitable documentation method would be a docstring. This is
because the function is simple and self-explanatory, and a docstring provides a clear and concise description of the function's purpose, its parameters, and its return value. Inline comments may not be necessary for such a straightforward function, and Google-style documentation may be too verbose for this case.
Example usage:

```
if __name__ == "__main__":
    original_string = "Hello, World!"
    reversed_string = reverse_string(original_string)
    print(f"Original string: {original_string}")
    print(f"Reversed string: {reversed_string}")
```

OUTPUT:

```
PS D:\AI Assisted coding>  & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugpy-2025.19.2026021801-win32-x64\b
undled\libs\debugpy\launcher' '64036' '--' 'D:\AI Assisted coding\.vscode\9.5.py'
Original string: Hello, World!
Reversed string: !dlroW ,olleH
PS D:\AI Assisted coding>
```

Problem 2: Password Strength Checker
Consider the function:

```
def check_strength(password):
return len(password) >= 8
```

Task:
1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

CODE:

```
def check_strength(password):
    """
    Check the strength of a password.

    A password is considered strong if it contains
    at least 8 characters.

    Args:
        password (str): The password string to be evaluated.

    Returns:
        bool: True if the password length is 8 or more characters,
              False otherwise.
    """
    # Check if the password length meets the minimum requirement
    return len(password) >= 8

# Example usage
if __name__ == "__main__":
    # Sample password to test
```

```
    password = "my_secure_password"

    # Check password strength and display result
    if check_strength(password):
        print("The password is strong.")
    else:
        print("The password is weak.")
```

OUTPUT:

```
PS D:\AI Assisted coding>  & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugpy-2025.19.2026021801-win32-x64\b
undled\libs\debugpy\launcher' '52465' '--' 'D:\AI Assisted coding\.vscode\9.5-2.py'
The password is strong.
PS D:\AI Assisted coding>
```

Problem 3: Math Utilities Module
Task:
1. Create a module math_utils.py with functions:
o square(n)
o cube(n)
o factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

CODE:
Create a module math_utils.py with functions:square(n)cube(n)factorial(n)Generate docstrings automatically using AI tools.Export documentation as an HTML file.

```
# math_utils.py
def square(n):
    """
    Returns the square of a number.

    Args:
        n (int or float): The number to be squared.

    Returns:
        int or float: The square of the input number.
    """
    return n * n

def cube(n):
    """
    Returns the cube of a number.

    Args:
        n (int or float): The number to be cubed.

    Returns:
        int or float: The cube of the input number.
    """
    return n * n * n
def factorial(n):
```

```
    """Returns the factorial of a number.

    Args:
        n (int): The number to calculate the factorial of. Must be a non-negative integer.
    Returns:
        int: The factorial of the input number.
    Raises:
        ValueError: If n is a negative integer.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    elif n == 0 or n == 1:
        return 1
    else:
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

# Example usage
if __name__ == "__main__":
    print(f"Square of 3: {square(3)}")
    print(f"Cube of 3: {cube(3)}")
    print(f"Factorial of 5: {factorial(5)}")
#Export documentation as an HTML file
import pydoc
if __name__ == "__main__":
    # Generate HTML documentation for the math_utils module
    html_doc = pydoc.HTMLDoc().docroutine(square) + pydoc.HTMLDoc().docroutine(cube) +
pydoc.HTMLDoc().docroutine(factorial)

    # Save the HTML documentation to a file
    with open("math_utils_documentation.html", "w") as file:
        file.write(html_doc)
    print("Documentation has been exported to math_utils_documentation.html")
```

OUTPUT :

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugp
y-2025.19.2026021801-win32-x64\bundled\libs\debugpy\launcher' '58031' '--' 'D:\AI Assisted coding\9.5-3.py'
Square of 3: 9
Cube of 3: 27
Factorial of 5: 120
Documentation has been exported to math_utils_documentation.html
```